

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCHEMA ANALYSIS AND VIVA VOCE

Course Code:	DSA05	Course Title:	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	Assessment:	Tool 2 – Schema Analysis and Viva Voce
Weightage:		Total Marks:	35 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)			
Student Name:	Dharshini H	Reg. No.:	192324268
Section / Batch:	B.Tech AI & DS	Date of Submission:	05-08-2026
Faculty In-charge:	Dr. B.SHYAMALA GOWRI	Project Mode:	Individual Submission

Code of Conduct

I Dharshini H 192324268 (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Dharshini H

Identification of Entities and Attributes (5 Marks)	Understanding of Relationships (5 Marks)	Analysis of Keys and Constraints (5 Marks)	Detection of Design Issues (10 Marks)	Viva Voce (5 Marks)	Total (35 Marks)

3. Online Shopping System

Database tables include:

- Customer(CustomerID, Name)
- Product(ProductID, ProductName, Price)
- Order(OrderID, CustomerID, OrderDate)
- OrderDetails(OrderID, ProductID, Quantity)

Each order may contain multiple products.

Viva Questions

Q1. Why is the OrderDetails table required in this schema?

Q2. What would happen if ProductID were stored directly in the Order table?

1. Identification of Entities and Attributes

Entity	Attributes
Customer	CustomerID (PK), Name
Product	ProductID (PK), ProductName, Price
Order	OrderID (PK), CustomerID (FK), OrderDate
OrderDetails	OrderID (PK, FK), ProductID (PK, FK), Quantity

2. Understanding of Relationships

1. Customer → Order

- Relationship: **One-to-Many (1:M)**
- One customer can place many orders.
- Each order belongs to one customer.

2. Order → OrderDetails

- Relationship: **One-to-Many (1:M)**
- One order can contain multiple order detail records.

3. Product → OrderDetails

- Relationship: **One-to-Many (1:M)**
- One product can appear in many order detail records.

4. Order ↔ Product

- Relationship: **Many-to-Many (M:N)**
- Implemented using the **OrderDetails** table.
- One order contains many products, and one product can be included in many orders.

3. Analysis of Keys and Constraints

Primary Keys (PK)

- Customer → **CustomerID**
- Product → **ProductID**
- Order → **OrderID**
- OrderDetails → **(OrderID, ProductID)** (Composite Primary Key)

Foreign Keys (FK)

- Order.CustomerID → Customer.CustomerID
- OrderDetails.OrderID → Order.OrderID
- OrderDetails.ProductID → Product.ProductID

Constraints

- CustomerID, ProductID, OrderID must be **unique**.
- Quantity should be **greater than 0**.
- Price should be **greater than or equal to 0**.
- CustomerID in Order must exist in Customer.
- ProductID and OrderID in OrderDetails must exist in their respective tables.

4. Detection of Design Issues

Existing Design

The database is generally **well normalized (3NF)** because:

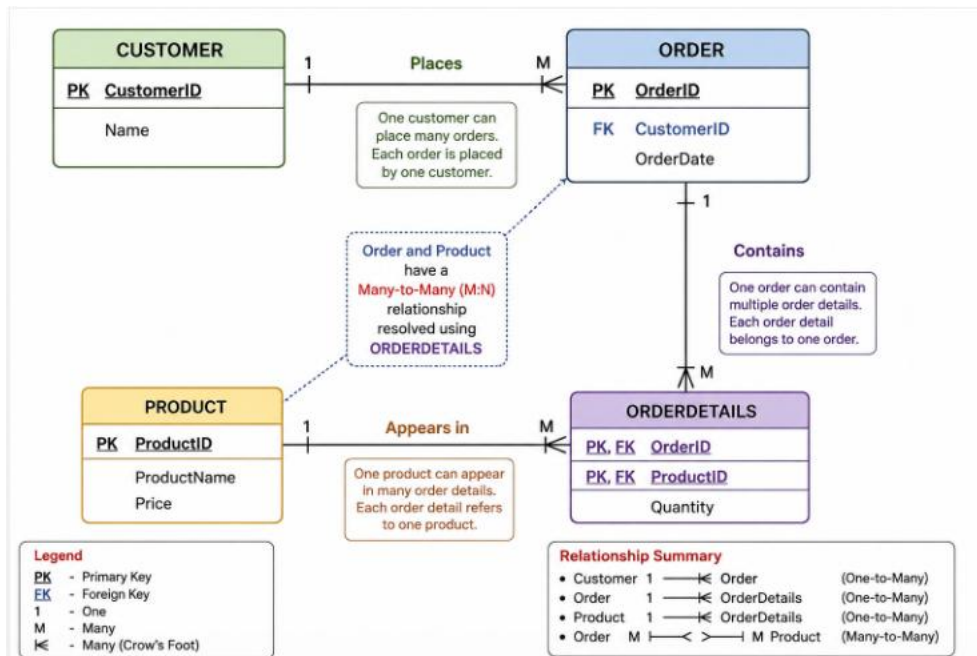
- Customer details are stored separately.
- Product details are stored separately.
- Orders are separated from products.
- Many-to-many relationship is resolved using OrderDetails.

Possible Design Issues and Improvements

1. **Customer table**
 - Only Name is stored.
 - Missing attributes like Email, Phone Number, Address.
2. **Product table**
 - Stock quantity is not maintained.
 - Product category is missing.
3. **Order table**
 - No Order Status (Pending, Shipped, Delivered).
 - No Payment Method or Payment Status.
4. **OrderDetails table**
 - Stores Quantity only.
 - Better to include **UnitPrice** at the time of purchase because product prices may change later.
5. **No Shipping Information**

- Shipping address and delivery details are not stored.
6. **No Discount or Tax Information**
- Cannot calculate the final bill accurately.

5. ER Relationship



Q1. Why is the OrderDetails table required in this schema?

The **OrderDetails** table is required to resolve the **many-to-many (M:N)** relationship between **Order** and **Product**.

- One order can contain **multiple products**.
- One product can appear in **multiple orders**.
- The **OrderDetails** table stores the products included in each order along with the **Quantity** of each product.

Without this table, it would not be possible to store multiple products for a single order efficiently.

Q2. What would happen if ProductID were stored directly in the Order table?

If **ProductID** were stored directly in the **Order** table, several design problems would occur:

- An order could store **only one product**, making it impossible to include multiple products in a single order.

- To store multiple products, the same **OrderID** would have to be repeated multiple times, leading to **data redundancy**.
- The database would suffer from **update, insertion, and deletion anomalies**.
 - The design would violate **database normalization principles**, making the schema inefficient and difficult to maintain.

10. Bus Ticket Reservation System

Tables:

- Passenger(PassengerID, Name)
- Bus(BusID, Route)
- Booking(BookingID, PassengerID, BusID, TravelDate)

Passengers can make multiple bookings.

Viva Questions

Q1. Why should booking details be stored separately from passenger information?

Q2. If a booking contains multiple passengers, how would you redesign the schema?

1. Identification of Entities and Attributes

Entity	Attributes
Passenger	PassengerID (PK), Name
Bus	BusID (PK), Route
Booking	BookingID (PK), PassengerID (FK), BusID (FK), TravelDate

2. Understanding of Relationships

1. Passenger → Booking

- Relationship: One-to-Many (1:M)
- One passenger can make multiple bookings.
- Each booking belongs to one passenger.

2. Bus → Booking

- Relationship: One-to-Many (1:M)
- One bus can have many bookings.
- Each booking is for one bus.

3. Analysis of Keys and Constraints

Primary Keys (PK)

- Passenger → PassengerID
- Bus → BusID
- Booking → BookingID

Foreign Keys (FK)

- Booking.PassengerID → Passenger.PassengerID
- Booking.BusID → Bus.BusID

Constraints

- PassengerID, BusID, and BookingID must be unique.
- PassengerID in Booking must exist in Passenger.
- BusID in Booking must exist in Bus.
- TravelDate should contain a valid date.

4. Detection of Design Issues (10 Marks)

Existing Design: The schema is suitable when one booking is made by one passenger.

Possible Design Improvements

1. Passenger table lacks details such as Phone Number, Email, and Address.
2. Bus table does not include Bus Number, Capacity, Driver, or Fare.
3. Booking table does not store Seat Number, Booking Status, or Payment Status.
4. There is no table for Bus Schedule (Departure Time, Arrival Time).
5. There is no cancellation or refund information.
6. If multiple passengers travel under one booking, the current design cannot support it.

Viva Questions

Q1. Why should booking details be stored separately from passenger information?

Booking details should be stored separately because a passenger can make multiple bookings over time.

Keeping booking information in a separate table:

- Avoids duplication of passenger details.
- Reduces data redundancy.
- Makes the database easier to maintain.
- Supports normalization by separating passenger information from booking transactions.

Q2. If a booking contains multiple passengers, how would you redesign the schema?

The schema can be redesigned by introducing a BookingPassenger table to handle the many-to-many relationship between Booking and Passenger.

Redesigned Schema

- Passenger(PassengerID, Name)
- Bus(BusID, Route)
- Booking(BookingID, BusID, TravelDate)
- BookingPassenger(BookingID, PassengerID)

Relationships

- One Booking → Many BookingPassenger records.
- One Passenger → Many BookingPassenger records.

This redesign allows:

- One booking to include multiple passengers.
- One passenger to appear in multiple bookings.
- A normalized and scalable database design.